# Practical: Neural Generative Models in Practice

## Edinburgh Summer School on Generative AI for Extremes

THE UNIVERSITY *of* EDINBURGH


| | |
|---|---|
| **Lead** | Johnny Lee; University of Edinburgh |


| | |
|---|---|
| **Date** | 8 September 2026 |
| **Gitub Repository** | `https://github.com/gamex-network/gamex_summer_school` |
| **Softwares** | `Python 3.11+`; `PyTorch 2.10+`; `NumPy`; `Matplotlib`; `JupyterLab`, `Git` |
| **Runtime** | 90 minutes |

| | |
|---|---|
| **Materials** | `01_gamex_transformer_edinburgh_airport.ipynb`<br>`02_gamex_normalising_flows_edinburgh_airport.ipynb` |
| **Before the session** | Please bring a laptop with `Python`, `PyTorch`, `NumPy`, `Matplotlib`, `JupyterLab` and `Git` preinstalled. Use of `Anaconda` or `Miniforge` is optional. A CPU is sufficient throughout the session and GPU is optional. |
| **Outputs** | A trained conditional autoregressive transformer model, a trained conditional normalising flow model, diagnostics plots, invertibility tests, generated trajectories and reusable `PyTorch` checkpoints. |

## Session Aims

1. Familiarise with `Python` syntax and `PyTorch` workflow behind neural generative modelling: tensors, `DataLoader`, `nn.Module`, `optim.AdamW` and `nn.functional`;
2. Implement and train a small **conditional autoregressive transformer** using causal attention mask and next location likelihood;
3. Implement and train a **conditional normalising flow** from affine coupling layers, including inverse maps and log-Jacobian determinants;

## Recommended Reading

- The session on *On Neural Generative Models* by M. de Carvalho
- Python Crash Course: Hands-on Project-based Introduction to Programming by E. Matthes [2]
- Deep Generative Modeling by J. Tomczak [5].

## Software Setup

A local `JupyterLab` installation is the recommended setup. The notebook automatically loads CUDA when available, Apple's MPS backend on compatible Macs, and otherwise falls back to CPU as device.

```
git clone https://github.com/gamex-network/gamex_summer_school.git
cd gamex_summer_school

python -m venv gamex
source gamex/bin/activate       # Windows PowerShell: venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install torch numpy matplotlib jupyterlab
```

`Run the first cell of each notebook before the session`. It prints the `PyTorch` version and selected device.

## Required Packages

| Package | Main use | Notes |
|---|---|---|
| `torch` | Tensor computation, neural network construction and optimisation | Provides `Tensor`, `DataLoader`, `nn.Module`, embeddings, Transformer layers, affine-coupling networks, backpropagation and `optim.AdamW`. |
| `numpy` | Numerical arrays and reproducibility | Used for auxiliary array operations, random-number handling and conversion between NumPy arrays and PyTorch tensors. |
| `matplotlib` | Visualisation and model diagnostics | Used to display aircraft trajectories, training and validation losses, latent-space diagnostics and comparisons between observed and generated samples. |
| `jupyterlab` | Interactive notebook environment | Provides the environment for running the practical notebooks and also installs `IPython` for interactive display. |

## Data and Files

**Notebook 1** `01_gamex_transformer.ipynb`. Discretises each aircraft trajectory into spatial tokens and trains a conditional autoregressive Transformer to model next-waypoint probabilities and generate trajectory continuations. The checkpoint `edinburgh_arrival_tf.pt` stores the trained model.

**Notebook 2** `02_gamex_normalising_flows.ipynb`. Uses the same continuous 30-waypoint arrivals, but represents each complete trajectory as a single 60-dimensional vector. The checkpoint `edinburgh_arrival_nf.pt` stores the trained model.

**Data** Both notebooks use the same synthetic airport arrival trajectories, conditioned on runway direction, so no external API, authentication or data download is required during the practical.

## PyTorch code skeleton

The two learned models share the same framework-level structure:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = my_model(...).to(device)
optimizer = torch.optim.AdamW(model.parameters(),
                              lr=...,
                              weight_decay=...,)

for epoch in range(1, n_epochs + 1):
    for inputs, targets in train_loader:
        model.train()
        optimizer.zero_grad()
        loss = loss_function(model, inputs, targets)
        loss.backward()
        optimizer.step()

model.eval()
with torch.inference_mode():
    generated = model_sampling(...)
```

- `my_model(...)` represents the model architecture defined using `nn.Module`; in this practical it is either an autoregressive transformer or a conditional normalising flow.
- `loss_function(...)` represents the model-specific negative log-likelihood used for training.
- `model_sampling(...)` represents the procedure for generating new samples after training.

## Key Details

Let $\boldsymbol{y} = (y_1, \ldots, y_N)$ denote an ordered sequence. An **autoregressive transformer** [6] represents the joint distribution through the chain rule,

$$p(\boldsymbol{y}) = \prod_{t=1}^{N} p(y_t \mid y_{<t}).$$

Each observation is mapped to a hidden representation and processed using causal masked multi-head self-attention, so that the representation at position $t$ depends only on the available past. Several transformer blocks are stacked to form richer sequence representations, which are then mapped to the parameters of the next step conditional distribution.

A **normalising flow** [4] instead treats the complete sequence as one vector and defines an invertible transformation $\boldsymbol{z} = f(\boldsymbol{y})$, where $\boldsymbol{z}$ follows a base distribution $\mathrm{N}(\mathbf{0}, \boldsymbol{I})$. By the change of variables,

$$\log p(\boldsymbol{y}) = \log p_Z(\boldsymbol{z}) + \log \left| \det \frac{\partial f(\boldsymbol{y})}{\partial \boldsymbol{y}} \right|.$$

In affine coupling flows [1], one subset of the variables is used to determine scale and shift transformations of the remaining variables. Several coupling layers with different partitions are composed to obtain a flexible invertible map.

Generation proceeds by drawing new samples from the probability distribution learned by the model. The precise sampling mechanism depends on the model structure. In an autoregressive model, observations are generated sequentially from the conditional distributions $p(y_t \mid y_{<t})$, with each sampled value appended to the sequence before generating the next. In a normalising flow, a complete latent vector $\boldsymbol{z}$ is first sampled from the base distribution and then transformed back to the data space through the inverse map, $\boldsymbol{y} = f^{-1}(\boldsymbol{z})$.

Both models define tractable probability distributions over sequences, but they achieve this differently: the autoregressive transformer factorises the distribution into ordered conditional distributions, whereas the normalising flow models the complete sequence through an invertible transformation with a tractable Jacobian determinant.

## Exercises

### Exercise 1: Autoregressive Transformer

1. Inspect the spatial tokenisation and shifted input–target pairs, and relate them to the factorisation $p(\boldsymbol{y} \mid \mathrm{RWY}) = \prod_t p(y_t \mid y_{<t}, \mathrm{RWY})$.
2. Identify the `PyTorch` training skeleton—`DataLoader`, `nn.Module`, `optim.AdamW`, loss, `backward()` and parameter update—and trace how one mini-batch moves through it.
3. Train the model and compare training and validation cross-entropy loss, then assess whether likelihood alone reflects the geometric quality of generated trajectories.
4. Generate the trajectories of the runways $p_{\boldsymbol{\theta}}(\boldsymbol{y} \mid \mathrm{RWY}) = \prod_t p_{\boldsymbol{\theta}}(y_t \mid y_{<t}, \mathrm{RWY})$ and vary temperature/top-$k$, then affect different scenarios.

### Exercise 2: Conditional Normalising Flow

1. Inspect the conditional affine coupling layer and verify that $f_{\boldsymbol{\theta}}^{-1}\{f_{\boldsymbol{\theta}}(\boldsymbol{x}; \mathrm{RWY}); \mathrm{RWY}\} \approx \boldsymbol{x}$.
2. Check that the forward and inverse log-Jacobian determinants cancel, as required by the change-of-variables construction.
3. Train the flow and compare its test NLL with the conditional Gaussian baseline to assess whether nonlinear dependence is being captured.
4. Sample $\boldsymbol{z} \sim \mathrm{N}(\mathbf{0}, \boldsymbol{I})$, generate $\boldsymbol{x} = f_{\boldsymbol{\theta}}^{-1}(\boldsymbol{z}; \mathrm{RWY})$ for both runway conditions, and compare likelihood-based and geometric diagnostics.

## If You Fall Behind

The practical notebooks have been pre-run and include representative outputs; students may rerun the cells during the session. The goal is not to type every line from memory, but to connect the probabilistic models to the code that makes it tractable.

1. **Notebook 1:** if model construction takes too long, run the provided transformer class and focus on the spatial tokens, shifted targets, causal mask, training loop and generation. Alternative decoding settings and prefix completion are extensions.
2. **Notebook 2:** prioritise one coupling layer, the full-flow round-trip test, exact NLL, training and sampling. The paired-latent runway experiment is optional.
3. We encourage the use of genAI tools such as `Cursor` and `Copilot` with responsibility and caution for helps such as auto completion or syntax error detection.

## Questions for Discussion

- What information is lost when the transformer converts continuous aircraft positions into spatial cell tokens? If you were to consider continuous input, how would you redesign the architecture?
- Why can the autoregressive transformer evaluate all target positions in parallel during training but not during autoregressive generation?
- Why does the flow require invertibility, whereas the conditioner inside a coupling layer does not?
- Which trajectory diagnostics would you require before trusting generated paths for a scientific or operational application?
- How can we use the pre-trained models [3] to generate arrival trajectories for other airport?

## References


[1] L. Dinh, J. Sohl-Dickstein, and S. Bengio. Density estimation using real NVP. *arXiv preprint arXiv:1605.08803*, 2016.

[2] E. Matthes. *Python Crash Course: A Hands-on Project-based Introduction to Programming*. no starch press, CA, 2019.

[3] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. Improving language understanding by generative pre-training. *OpenAI*, 2018.

[4] D. Rezende and S. Mohamed. Variational inference with normalizing flows. In *International Conference on Machine Learning*, pages 1530–1538. Proceedings of Machine Learning Research, 2015.

[5] J. M. Tomczak. *Deep Generative Modeling*. Springer Cham, 2nd edition, 2024.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.